GFT

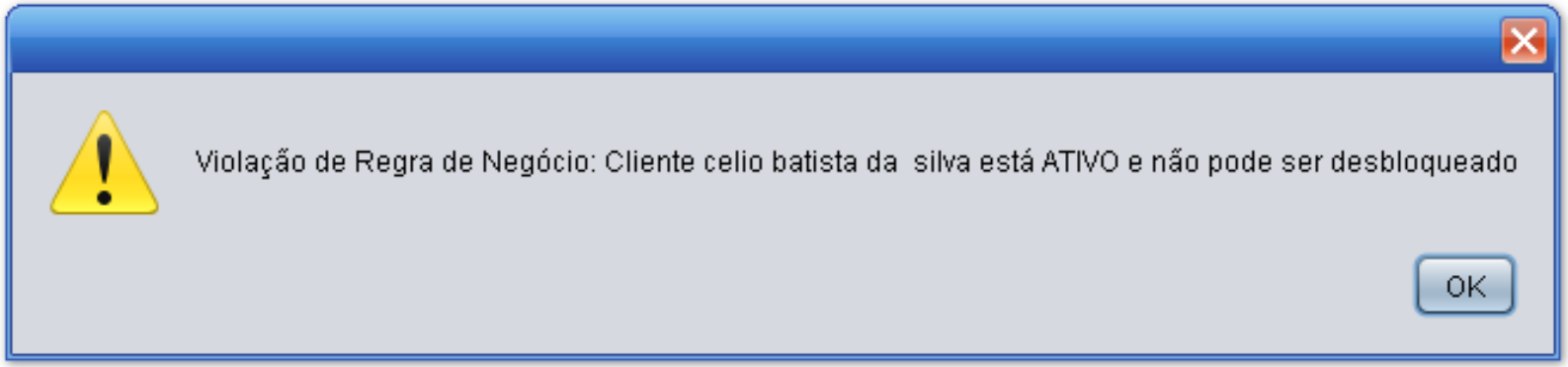
Java na Prática – Java e OO

Célio Silva

Aula 7

Java na Prática – Requisitos

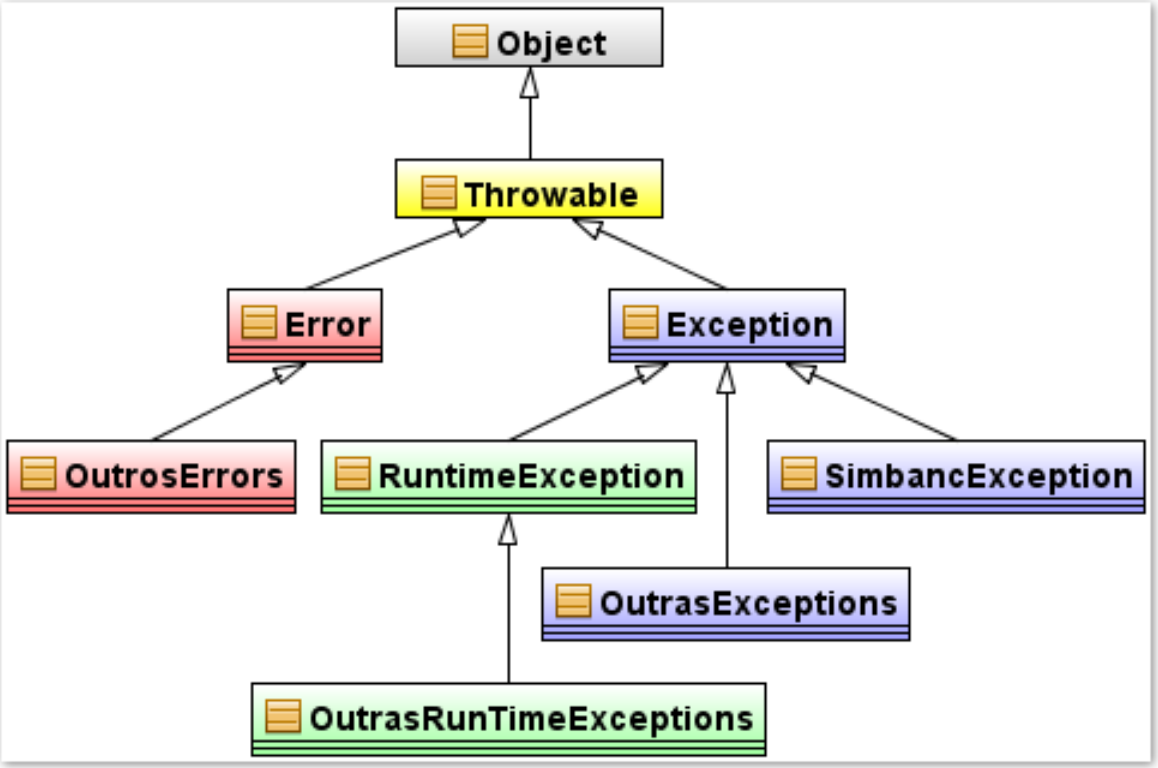
- Implementar regras para validar o estado de um cliente:
  - Somente clientes ATIVOS podem ser alterados
  - Somente clientes BLOQUEADOS podem ser desbloqueados
  - O e-mail pode ser nulo, mas deve estar em um formato correto a@a.a(.a) (Cadastrar e Alterar)



- Exercício
  - Implementar tratamento das regras de negócio acima através de exceções

Java na Prática – Tipos de Erro

- Throwable
  - `throw`
- Error
- Exception "*Checked*" - tratamento obrigatório
- RuntimeException *"Unchecked"* - tratamento opcional



- Exercício
  - Criar hierarquia:
  - `MinhaException`
    - `RegraNegocioExc..`
    - `SistemaException`

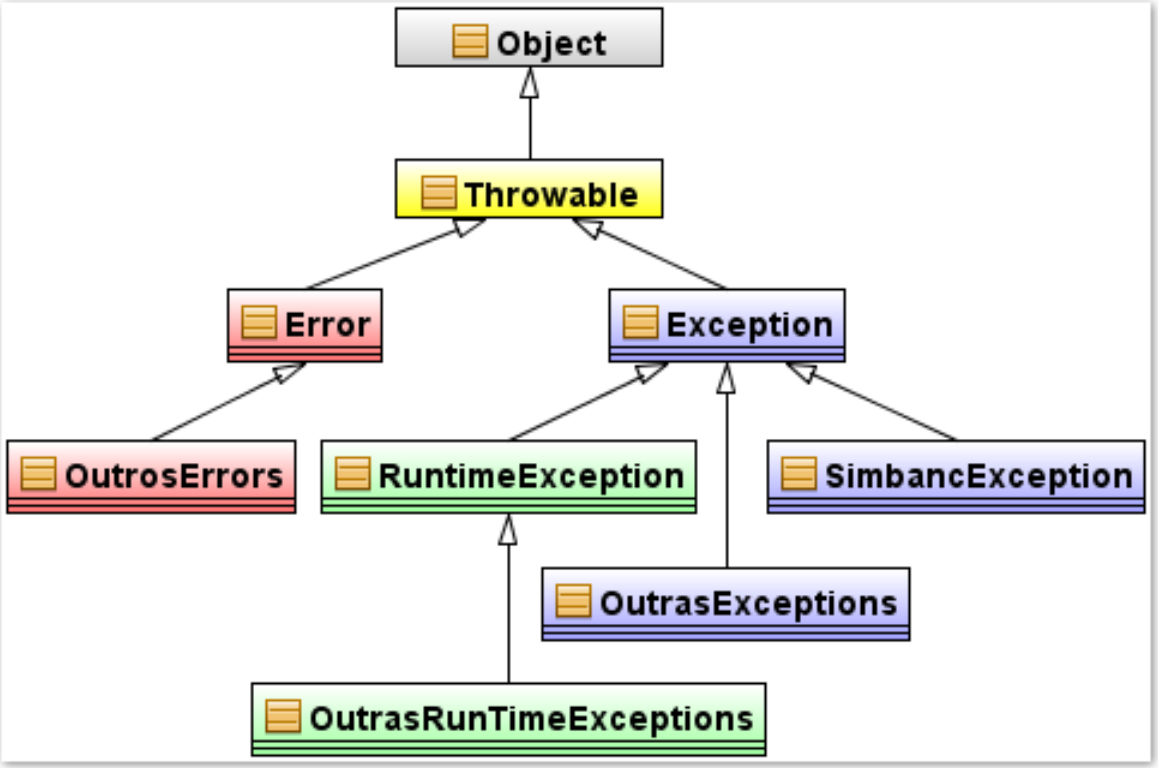
Java na Prática – java.lang.Exception – Regras

GFT

- Existem dois tipos de exceções: verificadas e não verificadas.
- Exceções verificadas incluem todos os subtipos de exceções, incluindo classes que herdem de RuntimeException
- Exceções verificadas são regidas pela regra tratar ou declarar; qualquer método que pode lançar uma exceção verificada (incluindo métodos que invocam métodos que podem lançar a exceção) devem também declarar a exceção usando throws ou tratá-la com um apropriado try/catch
- Subtipos de error ou RuntimeException não são verificados, então o compilador não lhe forçará a usar a regra de tratar ou declarar. Você está livre pra tratá-los ou para declará-los, mas o compilador não se importa se você está fazendo isso ou não.

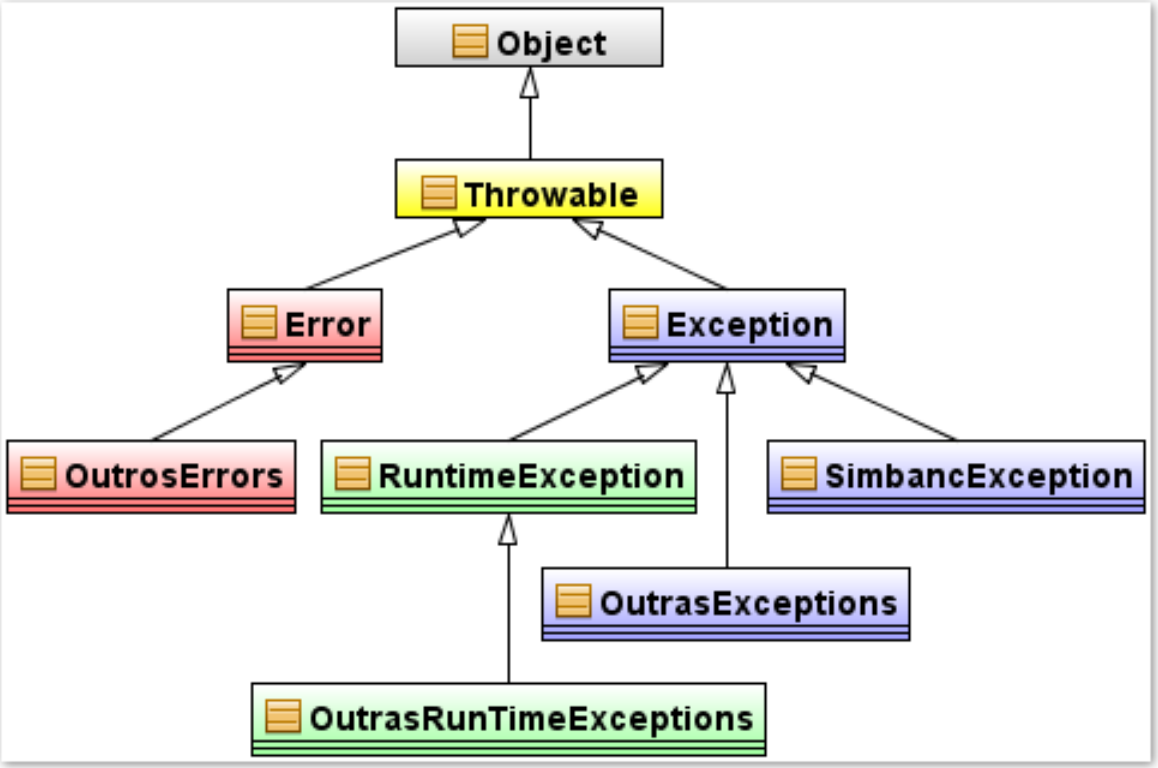
Java na Prática – java.lang.Exception – Principais Métodos

- `super()`
- `super(String message);`
- `super(String message, Throwable cause);`
- `super(Throwable cause);`
- `initCause(Throwable cause);`
- `getCause();`
- `getMessage();`
- `printStackTrace();`



Java na Prática – java.lang.Exception – Hierarquia de Exceções

- OutrosErrors
  - OutOfMemoryError
  - NoDefClassFoundError
- OutrasExceptions
  - IOException
  - SQLException
- OutrasRunTimeExceptions
  - NullPointerException
  - NumberFormatException



Java na Prática – java.lang.Exception – Tratamento Exceções

- "Exceptional Condition"
- Apanhando uma exceção

```
try {
} catch (Exception e){
    e.printStackTrace();
}
```

- Usando finally

```
try {
} catch (Exception e){
} finally {
    //liberar recursos, fechar conexões, etc
}
```

- Propagando exceções não apanhadas
  - `public Sucesso cadastrarCliente throws SimbancException {}`
- Apanhando exceções com polimorfismo
  - `"public void catch(Exception e){}"`

Java na Prática – Tratamento de Exceções

GFT

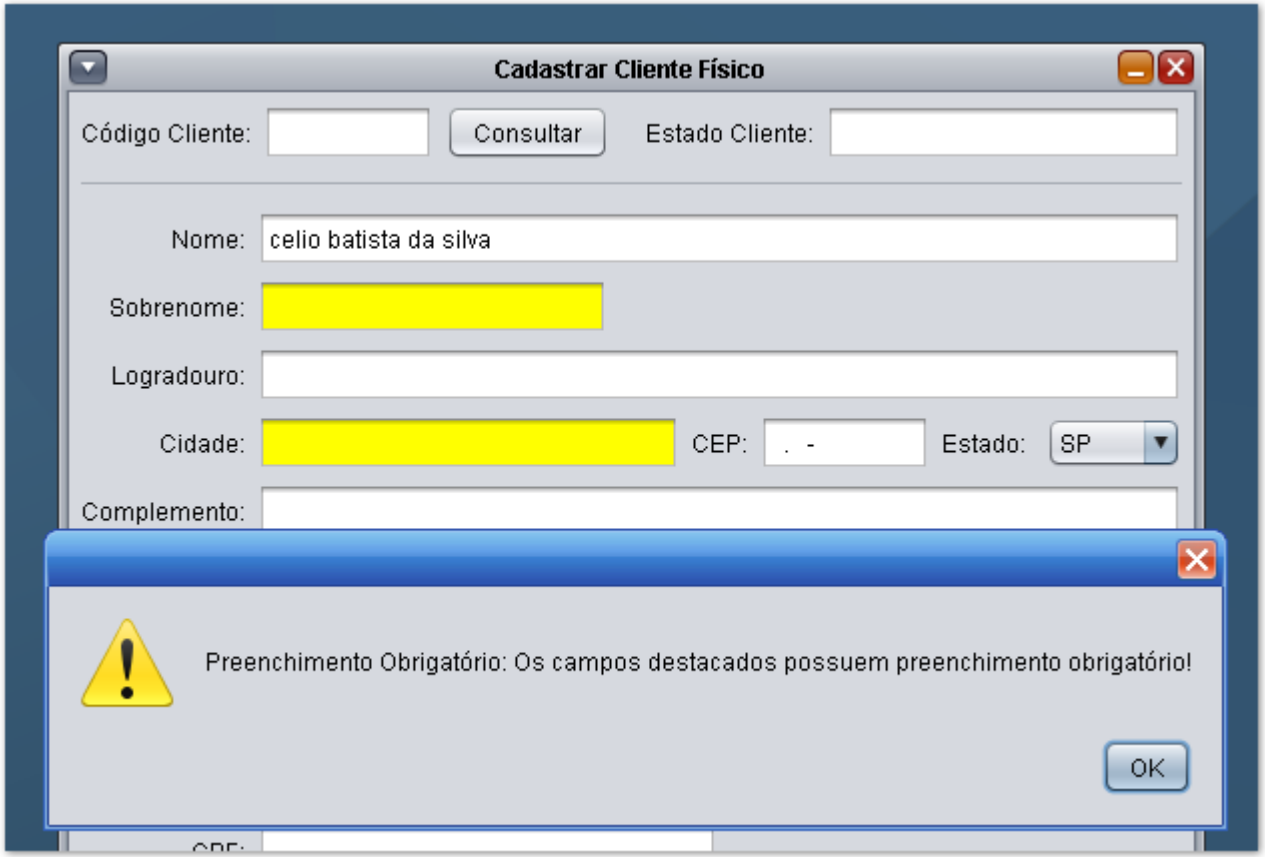
- A única exceção pela qual a regra "finally sempre será chamado" é que o finally não será chamado se a JVM desligar. Isto pode acontecer se o código de um bloco try/catch chamar System.exit()
- Se você usar um bloco finally opcionalmente, ele sempre será invocado, não importando se a execução no try correspondente será lançada ou não, e sem se importar se a exceção lançada é apanhada ou não
- Apenas porque finally é invocado não significa que ele terminará. Código no bloco finally pode também lançar uma exceção ou chamar System.exit()
- Exceções não apanhadas propagam através da pilha de chamadas, iniciando do método onde a exceção é lançada e terminando com o primeiro método que tem o catch correspondente para esta exceção ou a JVM desligar, o que acontece se a exceção chegar até o main e este declará-la e não tratá-la
- Você pode criar suas próprias exceções, apenas herdando de Exception ou um de seus subtipos. Sua exceção será considerada verificada e você será forçado a utilizar a regra de declarar ou tratar
- Todos os blocos catch devem estar ordenados com as exceções mais específicas para as mais genéricas. Caso contrário será criado um erro de compilação, informado-lhe que aquela exceção nunca poderá ser alcançada.
- Alguma exceções são criadas por programadores, outras pela JVM.

Java na Prática – Exceções mais comuns

Exceção	Lançada quando...	Lançador
`IndexOutOfBoundsException`	Tenta acessar um índice que está fora dos limites do array.	Programação JVM
`ClassCastException`	Tenta fazer uma conversão [cast] entre classes que não possuem relação de herança	JVM
`IllegalArgumentException`	O método recebe um argumento que não esperado	Programação
`IllegalStateException`	O sistema não pode fazer a ação que foi solicitada, como usar um scanner depois de fechá-lo	Programação
`NullPointerException`	Tenta usar os membros de um objeto que não foi instanciado ou possui valor null	Programação JVM
`NumberFormatException`	Tenta converter uma string incorreta em um número	Programação
`AssertionError`	O teste de afirmação retorna false	Programação
`ExceptionInInitializerError`	Tenta inicializar uma variável static ou um bloco de inicialização	JVM
`StackOverflowError`	Um método recursivo é chamado muitas vezes	JVM
`NoClassDefFoundError`	A JVM não consegue encontrar uma classe ou arquivo necessário para executar o programa	JVM
`ConcurrentModificationException`	Modificação do mesmo objeto realizado em dois lugares distintos simultâneamente	Programação
`UnsupportedOperationException`	Quando uma operação foi herdada mas não foi implementada	Programação

Java na Prática – Requisitos

- *Validar campos que possuem preenchimento obrigatório e informar o usuário, com um aviso claro e nítido, que aquele campo não pode permanecer em branco.*
  - *Cliente Fisico – Nome, Sobrenome e Cidade*
  - *Cliente Jurídico – Nome, Cidade*



Java na Prática – Var-args

- public void fazerAlgumaCoisa(TipoVariavel... argumentos)
- Por for:
  - Possibilidade de passar múltiplos argumentos ou nenhum
- Por dentro:
  - Recebe um array do tipo de clarado.
  - O array irá conter todos os tipos declarados ou nenhum, mas nunca será nulo.
- A partir do Java 5
- Um método com var-arg somente pode conter um parâmetro var-arg
- Em métodos com parâmetros normais e um var-arg, o var-arg deve vir em último lugar

- Exercício
  - Implementar um validador de campos com var-args.

```
public static final void validarPreenchimentoObrigatorio(JTextField...
campos) throws PreenchimentoObrigatorioException
```

GFT

Referências Bibliográficas:

Certificação Sun para programador Java 5: guia de estudo (exame 310-055) / Kathy Sierra, Bert Bates

Effective Java: Second Edition/First Edition / Joshua Bloch

Sugestões, dicas ou reclamações: celio@celiosilva.com

Obrigado!

GFT Technologies AG

Av. São Francisco, 98 - STA Rosália, CEP 18095-450.

celio.da-silva@gft.com

T +55 15 3332-0762

Célio Batista da Silva

celio@celiosilva.com

© Copyright Célio Batista da Silva, 2010